Report

Qrup: 699.24

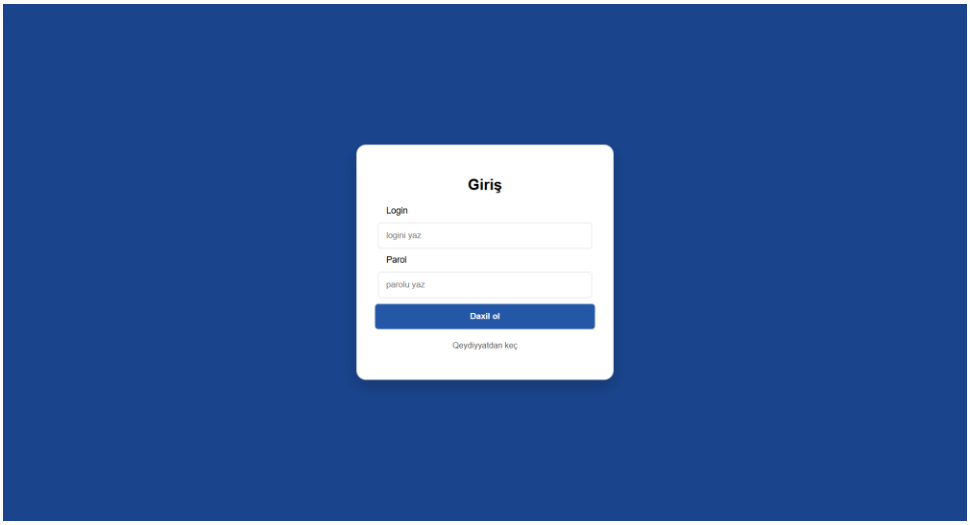
Fənn: Veb Təhlükəsizlik

Tələbə: Saleh Salmanlı

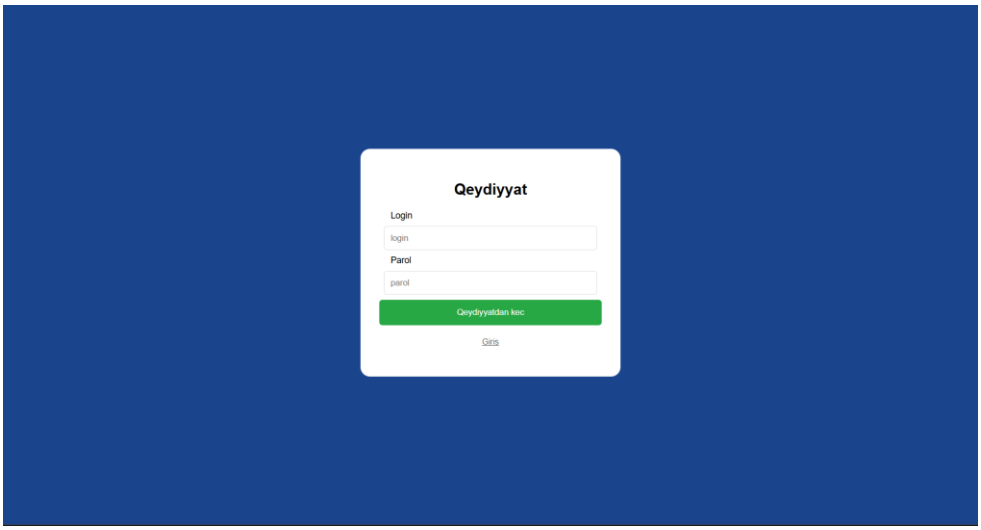
Giriş

Saytın ümumi görünüşü:

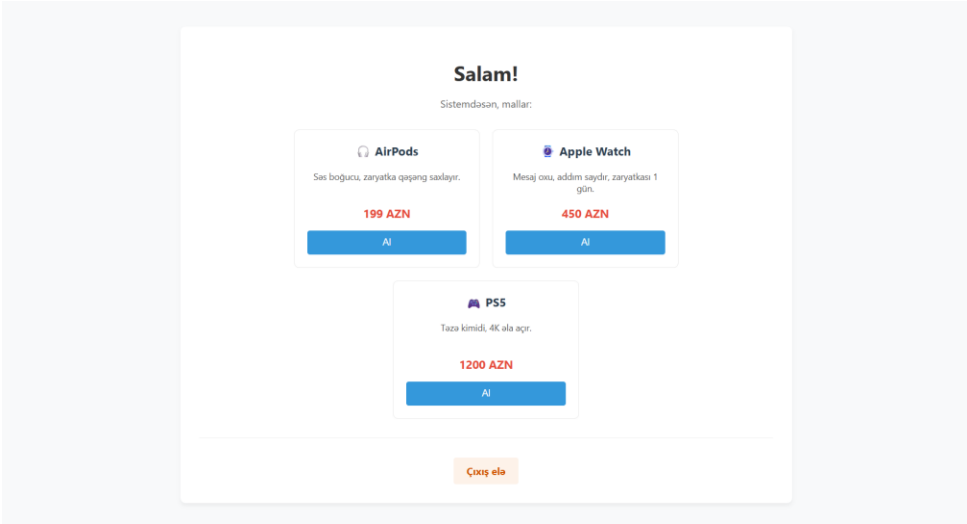
Login səyfəsi:



Register səyfəsi:



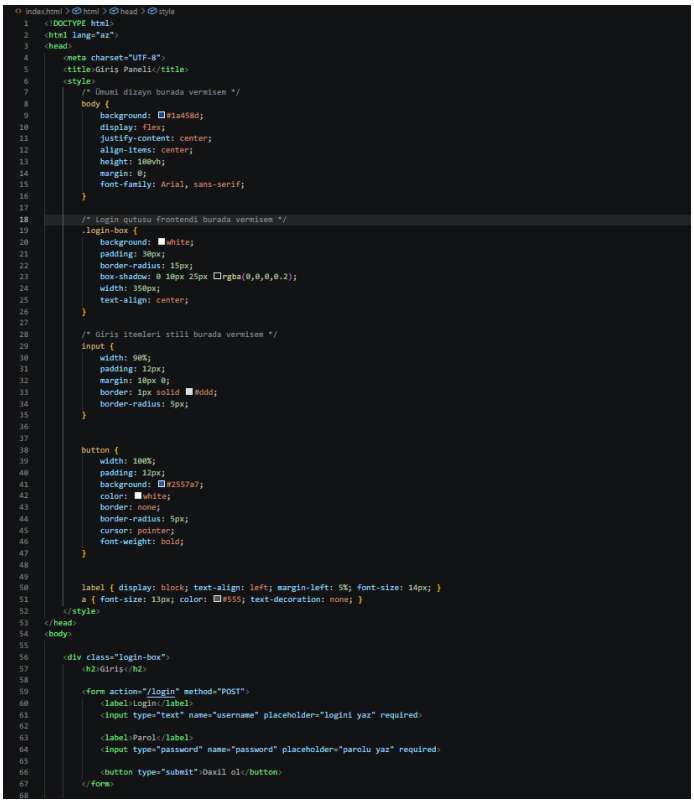
Əsas səyfə:



Açıqlama

Bu web sayt elektronik məhsullarının satışı üçün nəzərdə tutulmuşdur. Web Site – da 1) Login 2) Register 3) Əsas page var.

Login Page:



```
index.html > html > head > style
<!DOCTYPE html>
<html lang="az">
<head>
    <meta charset="UTF-8">
    <title>Giriş Paneli</title>
    <style>
        /* Ümumi dizayn burada vermisem */
        body {
            background: #1a458d;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
            font-family: Arial, sans-serif;
        }

        /* Login qutusu frontendi burada vermisem */
        .login-box {
            background: white;
            padding: 30px;
            border-radius: 15px;
            box-shadow: 0 10px 25px rgba(0,0,0,0.2);
            width: 350px;
            text-align: center;
        }

        /* Giriş itemləri stili burada vermisem */
        input {
            width: 90%;
            padding: 12px;
            margin: 10px 0;
            border: 1px solid #ddd;
            border-radius: 5px;
        }


        button {
            width: 100%;
            padding: 12px;
            background: #2557a7;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            font-weight: bold;
        }


        label { display: block; text-align: left; margin-left: 5%; font-size: 14px; }
        a { font-size: 13px; color: #555; text-decoration: none; }
    </style>
</head>
<body>

    <div class="login-box">
        <h2>Giriş</h2>

        <form action="/login" method="POST">
            <label>Login:</label>
            <input type="text" name="username" placeholder="login yaz" required>

            <label>Parol</label>
            <input type="password" name="password" placeholder="parolu yaz" required>

            <button type="submit">Daxil ol</button>
        </form>
```

Giriş qutusunu ekranın düz ortasına gətirmək üçün **Flexbox** (display: flex) funksiyasından istifadə eləmişəm çünki ekranı böyüdüb-kiçildəndə o qutu həmişə mərkəzdə qalır.

İşləmə Məntiqi

Əsas iş burada formanın daxilində gedir:

- **POST Metodu:** Məlumatları göndərəndə GET yox, məhz POST metodunu seçmişəm. Niyə? Çünki GET eləsəydim, istifadəçinin yazdığı parol yuxarıda, brauzerin ünvan sətrində hamıya görünərdi. POST isə bu məlumatları gizli şəkildə arxa tərəfdən göndərir.
- **Action ("/login"):** Bura "məlumatlar hara getsin?" sualının cavabıdır. Düyməni basanda brauzer avtomatik bilir ki, bu login və parolu götürüb bizim backend-dəki `/login` hissəsinə çatdırmalıdır.
- **Name Atributları:** `name="username"` və `name="password"` hissələrini qəsdən belə adlandırmışam. Çünki server tərəfi (bizim `server.js`) gələn paketin içində "username" və "password" etiketli məlumatları axtarır. Bu adlar olmasa, server nəyin login, nəyin parol olduğunu başa düşməz.

Backend:

```
26  // Giriş (Login) POST sorğusu
27  app.post('/login', (req, res) => {
28
29      const query = `SELECT * FROM users WHERE username = '${req.body.username}' AND password = '${req.body.password}'`;
30
31      // Sorğunu icra edilir ve nəticəni alırıq
32      const user = db.prepare(query).get();
33
34      // əgər true olarsa (yəni tapıldısa), panelə buraxırıq
35      if (user) {
36          res.redirect('/dashboard.html');
37      } else {
38          // Tapılmadısa xəta qaytarırıq
39          res.send("Sehvdir! <a href='/index.html'>Geri</a>");
40      }
41  });
42
```

1. Sorğunun Qarşılanması

Mən burada `app.post('/login', ...)` funksiyasını işlətmişəm. Bunun mənası odur ki, kimsə login səhifəsində düyməni basanda, onun məlumatları "paket" halında bura gəlir. `req` (request) həmin paketin özüdür, `res` (response) isə mənim istifadəçiyə qaytaracağım cavabdır.

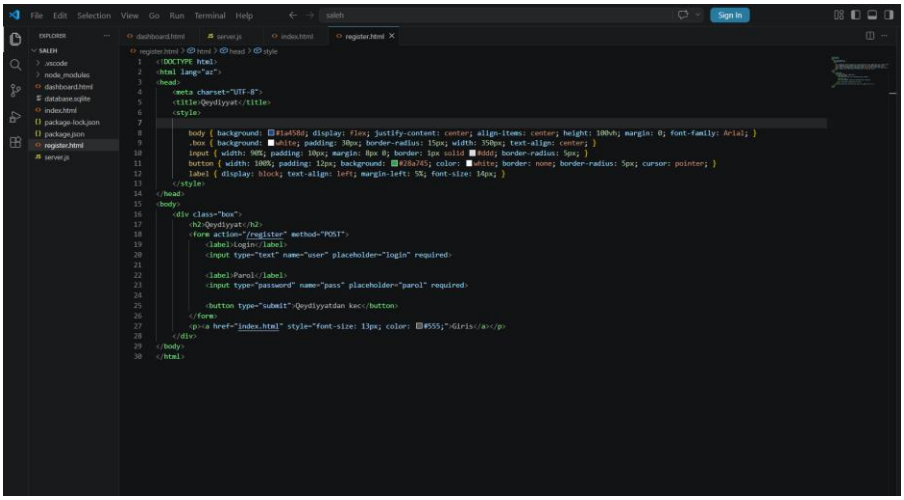
2. SQL Sorğusunun Hazırlanması (Kritik Hissə)

- **Məntiq:** Mən bazaya deyirəm ki: "GİT, `users` cədvəlindən elə bir adamı tap ki, onun `username`-i və `password`-u istifadəçinin yazdığı ilə eyni olsun".
  Mən burada `${req.body.username}` strukturundan istifadə edərək istifadəçinin yazdığı mətni birbaşa SQL sorğusunun içinə "yapışdırmışam".

3. Bazada Yoxlama və Yönləndirmə

- **İcra:** `db.prepare(query).get()` sətirində mən hazırladığım sorğunu bazada işə salıram və nəticəni `user` adlı dəyişənə yükləyirəm.
- **Uğurlu Giriş:** Əgər bazada belə bir adam varsa (`if (user)`), mən `res.redirect` funksiyasını çağırıram. Bu, istifadəçini avtomatik olaraq bizim malların olduğu `dashboard.html` səhifəsinə atır.
- **Uğursuz Giriş:** Əgər baza boş qayıtsa (yəni belə bir user yoxdursa), `else` hissəsi işə düşür və ekrana "Sehvdir!" yazısı ilə bir geri qayıtma linki çıxarıram.

Register Page:



1. Məlumat Bazasının Hazırlanması

- **Avtomatik Cədvəl Yaradılması:** `db.exec` funksiyasından istifadə edərək yoxlayıram ki, əgər bazada `users` cədvəli yoxdursa, proqram onu dərhal yaratsın.

2. Faylların Brauzerə Çatdırılması

- **Statik Fayllar:** `express.static` funksiyasını işlətməklə brauzerə icazə verirəm ki, mənim `index.html` və digər fayllarımı birbaşa oxuya bilsin.
- **Port Ayarı:** Serveri 8000-ci portda işə salmışam ki, lokal olaraq rahatlıqla test edə bilim.

3. Yeni İstifadəçinin Qeydiyyatı (`/register`)

Bu funksiyanı yeni adamların sistemə daxil ola bilməsi üçün yazmışam:

- **Məlumatın Tutulması:** İstifadəçi qeydiyyat formasını dolduranda, mən həmin datanı `req.body` vasitəsilə qəbul edirəm.
- **Bazaya Yazılma:** `INSERT INTO` əmri ilə həmin login və parolu bazaya əlavə edirəm. Burada `db.prepare` funksiyasını seçmişəm ki, məlumatlar bazaya düzgün və səliqəli şəkildə yerləşsin.
- **Yönləndirmə:** Qeydiyyat bitən kimi istifadəçini avtomatik olaraq giriş səhifəsinə qaytarıram ki, təzə hesabı ilə daxil ola bilsin.

4. Giriş Mexanizmi və Təhlükəsizlik Boşluğu (`/login`)

Layihəmin ən əsas və kritik hissəsi buradır. Burada həm giriş sistemini qurmuşam, həm də öyrəndiyimiz cybersecurity mövzularını tətbiq etmişəm:

- **SQL Sorğusu:** İstifadəçi giriş edəndə mən bazaya belə bir sual göndərirəm: "Bu login və parolda adam varmı?".
- **SQL Injection (Qəsdən qoyulan boşluq):** Mən burada istifadəçidən gələn məlumatları heç bir təmizləmə prosesindən keçirmədən birbaşa SQL sorğusunun daxilinə yerləşdirmişəm. Bu, bizim dərslərdə keçdiyimiz **SQL Injection** hücumuna tam şərait yaradır. Məqsədim bu boşluğun necə işlədiyini əyani göstərməkdir.
- **İdarəetmə:** Əgər baza "hə, belə bir istifadəçi var" desə, mən onu malların siyahısı olan `dashboard.html` səhifəsinə buraxıram. Tapılmasa isə "Sehvdir!" mesajı qaytarıram.

5. Frontend və Backend Arasındakı Əlaqə

Formları hazırlayarkən məlumatların itməməsi və düzgün çatması üçün bu metodları seçmişəm:

- **POST Metodu:** Həm login, həm də qeydiyyat formalarında POST metodunu tətbiq etmişəm ki, parollar ünvan sətrində açıq-aşkar görünməsin.
- **Data Etiketləri:** Formadakı xanalara `name="username"` və ya `name="user"` kimi adlar vermişəm ki, backend-də həmin məlumatları adları ilə rahatlıqla çağıra bilim.

```
// Qeydiyyat POST sorğusu
app.post('/register', (req, res) => {
    // Sırf login və parolu bazaya əlavə edirik (Ad formdan və bazadan silindi)
    const stmt = db.prepare('INSERT INTO users (username, password) VALUES (?, ?)');
    stmt.run(req.body.user, req.body.pass);

    // qeydiyatdan sonra login page gelir
    res.redirect('/index.html');
});
```

Backend:

1. Sorğunun Qəbulu

Burada `app.post('/register', ...)` funksiyasından istifadə etmişəm. Bu o deməkdir ki, kimsə qeydiyyat səhifəsində düyməni basanda, onun yazdığı məlumatlar paket halında məhz bu ünvana gəlir.

2. Məlumatların Bazaya Yazılması

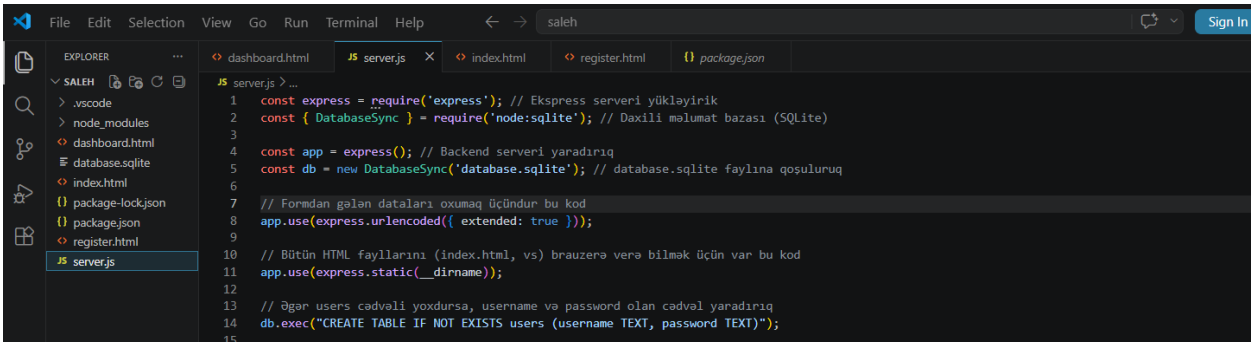
Hazırladığım sorğunu `stmt.run` funksiyası ilə işə salmışam.

- Burada `req.body.user` və `req.body.pass` dəyişənlərini ötürmüşəm.
- Bunlar birbaşa bizim `register.html` faylındakı xanaların içindəki mətnlərdir.
- Beləliklə, istifadəçinin yazdığı login və parol gedib `database.sqlite` daxilindəki `users` cədvəlinə oturur. Kodda da qeyd etdiyim kimi, burada sırf login və parolu yadda saxlayıram.

3. Giriş Səhifəsinə Yönləndirmə

Hər şey uğurla bitəndən sonra, mən istifadəçini `res.redirect('/index.html')` funksiyası ilə yenidən giriş səhifəsinə göndərirəm. Çünki qeydiyyatdan keçən adamın artıq növbəti addımı sistemə daxil olmaqdır və bunun üçün o, login panelini görməlidir.

Serverin Qurulması Üçün Lazımlı Alətlər



- **Express :** Burada mən `express` kitabxanasını layihəmə daxil etmişəm. Bu, mənim backend serverimin əsas karkasıdır. `const app = express()` yazaraq bütün sistemi bu dəyişən üzərindən idarə etməyə başlamışam.
- **SQLite:** Məlumatları yadda saxlamaq üçün Node.js-in daxili `node:sqlite` modulunu seçmişəm. `new DatabaseSync` kodu ilə birbaşa `database.sqlite` faylına qoşulmuşam. Əgər bu fayl yoxdursa, proqram onu avtomatik olaraq yaradır.

Məlumatların Qəbulu və Faylların Paylanması

Serverin daxil olan sorğuları düzgün başa düşməsi üçün bu iki funksiyanı əlavə etmişəm:

- **Form Məlumatları :** `app.use(express.urlencoded)` funksiyası login və ya qeydiyyat formasını doldurub göndərəndə, həmin məlumatlar serverə mürəkkəb formatda gəlir. Bu kod həmin dataları "tərcümə edir" ki, mən onları `req.body` vasitəsilə rahat oxuya bilim.
- **Statik Fayllar :** Mən burada `express.static` funksiyasını işlətmişəm. Bu o deməkdir ki, brauzer mənim qovluğumdakı `index.html`, `register.html` və digər dizayn fayllarını heç bir əlavə kod yazmadan birbaşa oxuya biləcək.

Məlumat Bazasının İlk Hazırlığı

- **Cədvəlin Yaradılması :** Bura `db.exec` funksiyası ilə bir SQL əmri yerləşdirmişəm. Mən proqrama deyirəm ki: "Yoxla, əgər bazada `users` adlı cədvəl yoxdursa, dərhal onu yarat".
- **Sütunlar:** Cədvəlin daxilində `username` və `password` adlı iki sütun təyin etmişəm.

Əlaqələr:

Backendi fronta baglayan:

```
<form action="/login" method="POST">
```

Mən burada backend-ə bir "qapı" açmışam. `action="/login"` hissəsi ilə deyirəm ki, məlumatlar serverdəki `/login` ünvanına getsin. `method="POST"` isə həmin məlumatları bir paketdə göndərir

Frontu Backende baglayan:

```
app.post('/login', (req, res) => {
```

Frontend-dən `/login` ünvanına bir məlumat gələn kimi bu funksiya işə düşür və gələn paketi açmağa başlayır.

Gelen datalari oxunan hala salan kod:

```
app.use(express.urlencoded({ extended: true }));
```

Backendin bazaya baglandigi yer:

```
stmt.run(req.body.user, req.body.pass);
```

bu sətir backend ilə bazanı birləşdirən son nöqtədir. Mən burada frontend-dən gələn `user` və `pass` məlumatlarını götürüb birbaşa `database.sqlite` faylındakı cədvələ yazıram.

Bazadan fronta gelen cavab:

```
    res.redirect('/dashboard.html');
```

Verilənlər bazasına göndərilən SQL sorğusu müsbət nəticə verdikdə, server tərəfindən HTTP yönləndirməsi icra olunur və istifadəçi `dashboard.html` interfeysinə göndərilir.

Portun idarə edilməsi:

```
app.listen(8000, () => {
    console.log("Server qosuldu");
});
```

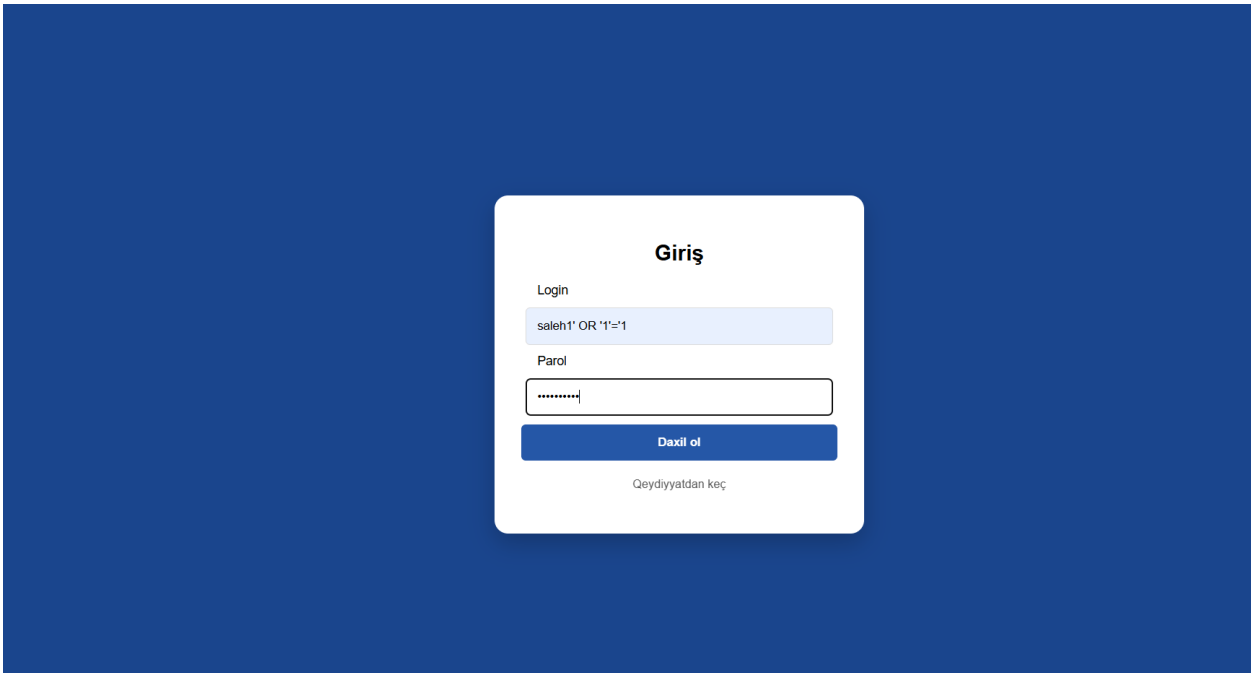
Bu funksiya backend serverinin işə salınmasını və təyin edilmiş TCP portu (8000) üzərindən daxil olan HTTP sorğularının qəbulunu təmin edir.

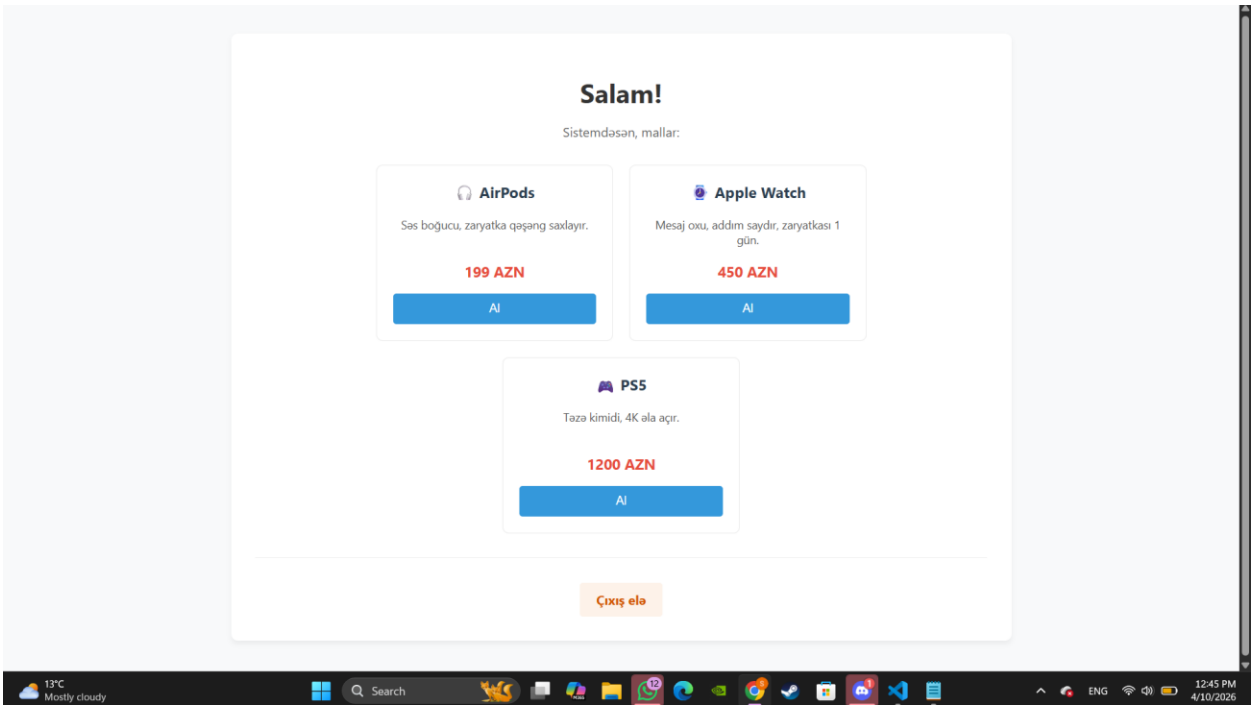
SQL Injection

SQL Injection — proqram təminatının məlumat bazasına göndərdiyi sorğulara kənardan müdaxilə edilməsi hücumudur. Bu hücum o zaman mümkün olur ki, proqramçı

istifadəçidən gələn məlumatları heç bir təmizləmə prosesindən keçirmədən birbaşa SQL əmrinə birləşdirir.

Login yerine "admin' OR '1'='1" yaziriq neticede parola ehtiyac duyulmadan esas seyfeye kecid edeceyik.





Bizim tehlukeli kodumuz ise budur:

```
29    const query = `SELECT * FROM users WHERE username = '${req.body.username}' AND password = '${req.b
```

Problem buradadır: Verilənlər bazası (SQLite) proqramçının yazdığı SQL əmrləri ilə istifadəçinin yazdığı mətni bir-birindən ayıra bilmir. Haker login xanasına `admin' OR '1'='1` yazdığı zaman, onun daxil etdiyi tək dırnaq (`'`) işarəsi sənin sorğundakı username sahəsini vaxtından əvvəl bağlayır. Bundan sonra gələn `OR '1'='1'` hissəsi isə artıq sadəcə bir mətn yox, SQL-in öz əmri (məntiqi şərti) kimi qəbul edilir.

Baza bu sorğunu bütöv şəkildə oxuyur və görür ki, proqramçı tərəfindən yazılmış parol yoxlamasından asılı olmayaraq, ortada həmişə doğru olan bir şərt (1=1) var. Beləliklə, haker bazanı "aldadaraq" giriş icazəsi alır.

```
// Təhlükəsiz Variant
const query = 'SELECT * FROM users WHERE username = ? AND password = ?';

// Məlumatlar sorğuya birbaşa deyil, ayrıca parametr kimi ötürülür
const user = db.prepare(query).get(req.body.username, req.body.password);
```

3. Bu Kod Niyə Təhlükəsizdir?

Bu metodun təhlükəsiz olmasının 3 əsas səbəbi var:

1. **Kod və Verilənlərin Ayrılması:** Biz SQL sorğusunun **strukturunu** (şablonunu) istifadəçidən gələn **məlumatdan** tamamilə ayırırıq. Sual işarələri (`?`) proqramın strukturunu qoruyan "yer tutucular"dır.
2. **Məlumatın "Data" kimi qəbul edilməsi:** Verilənlər bazası drayveri (`better-sqlite3`) sual işarələrinin yerinə gələn mətni heç vaxt "əmr" kimi oxumur. Haker oraya `OR '1'='1'` yazsa belə, baza bunu SQL kodu kimi yox, sadəcə bir nəfərin uzun və qəribə adı kimi qəbul edir. Bazada da belə bir adda istifadəçi olmadığı üçün hücum uğursuz olur.
3. **Avtomatik Təmizləmə (Escaping):** Parametr kimi ötürülən məlumatların içindəki dırnaq (`'`) və ya digər təhlükəli simvollar baza tərəfindən avtomatik olaraq "zərərsizləşdirilir". Yəni həmin simvolların SQL sorğusunun məntiqini dəyişmək gücü əlindən alınır.